Тема

Персонаж с приватными статами (замыкания) и управлением через DOM-кнопки.

Цель

1. Отработать замыкания: хранение приватного состояния внутри фабрики.
2. Отработать DOM: кнопки → методы → обновление интерфейса.
3. Научиться делать простую игровую логику: “тики”, ограничения, смерть.

Входные данные

Студенту выдаётся стартовый проект (или разметка), где уже есть:

1) HTML-элементы

```html
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Ваш персонаж</title>

    <link rel="stylesheet" href="styles.css">

</head>



<body>



    <div class="your-character">
```

```html
        <img class="character" src="./img/yourcharacter.png" alt="character">

    </div>



    <div class="container">

        <div class="states">
        <!-- Статистика -->

            <div class="state" id="water">

                <div class="name">Вода</div>
                <!-- Здесь может любая Ваша стата -->

                <span class="count"></span>

            </div>

            <div class="state" id="food">

                <div class="name">Еда</div>
                <!-- Здесь может любая Ваша стата -->

                <span class="count"></span>

            </div>

            <div class="state" id="energy">

                <div class="name">Энергия</div>
                <!-- Здесь может любая Ваша стата -->

                <span class="count"></span>

            </div>



        </div>



        <div class="actions">
        <!-- Действия -->
```

```html
        <button class="action" id="drink">Выпить</button>

        <button class="action" id="eat">Поесть</button>

        <button class="action" id="rest">Отдохнуть</button>

      </div>



      <div class="alert">Перезагрузите страницу, чтобы начать сначала</div>
    <!-- Алерт, если персонаж умрет -->


   </div>
   <script src="script.js"></script>
</body>
</html>
```

2) Ограничения по разработке

- Код запускается после `DOMContentLoaded`.
- Запрещено хранить статы персонажа в глобальных переменных (только внутри фабрики).
- Диапазон статов: **0..100** (если не указано иначе).

На 4 (обязательная часть)

1) Придумать своего персонажа

Студент выбирает персонажа и оформляет его в комментарии в начале файла:

Пример:

- Имя: “Наруто” / “Геральт” / “Сатору” / “Антропий” / “Кот-хакер”
- Короткое описание (1–2 строки)
- Статы в этой работе остаются **всегда**: вода/еда/энергия (water/food/energy)

> На 4 НЕ требуется конфиг. Достаточно выбрать персонажа и назвать его в комментарии.

2) Фабрика персонажа (замыкание)

Реализовать функцию:

```
function createCharacter(initialState = 100) {  ...  }
```

Она должна:

- создать приватное состояние внутри (`water` , `food` , `energy`)
- вернуть объект методов:

Методы:

- `getWater()` , `getFood()` , `getEnergy()` → возвращают числа
- `drink(amount = 10)` → увеличивает water
- `eat(amount = 10)` → увеличивает food
- `rest(amount = 10)` → увеличивает energy
- `tick()` → уменьшает все статы (например: water -8, food -7, energy -6)

❗ Важно: методы должны менять **состояние**, а UI только отображать.

3) Clamp (0..100)

Любое изменение статов должно быть ограничено диапазоном 0..100:

- меньше 0 → 0
- больше 100 → 100

4) Таймер “жизни”

Каждую секунду:

- вызывается `tick()`
- обновляются значения в UI (`textContent`)
- если значение ≤ 30 — цвет показателя красный, иначе чёрный

5) Смерть персонажа

Если любой стат стал равен 0:

- остановить таймер (clearInterval)
- отключить кнопки действий
- показать `.alert`
- вывести confirm “Начать заново?” и при согласии перезагрузить страницу

✅ Сдача на 4: персонаж живёт/умирает, статы приватные (замыкание), UI обновляется.

Оценка 5

Нужно реализовать **минимум 2** пункта из списка.

Усложнение A. Конфиг персонажа (настройка через объект)

Переделать фабрику так, чтобы она принимала конфиг:

```
createCharacter(config)
```

Конфиг задаёт:

- `name` (имя)
- `stats` (стартовые значения)
- `decay` (убывание в tick)
- `actions` (силы действий)

Примеры конфигов (оставлены 2)

1) Антропий

```
const ANTROPY = {
  name: "Антропий",
  stats:   { water: 100, food: 100, energy: 100 },
  decay:   { water: 8,   food: 7,   energy: 6 },
  actions: { drink: 15,  eat: 12,   rest: 10 }
};
```

2) Кибер-страж

```
const CYBER_GUARD = {
  name: "Кибер-страж",
  stats:   { battery: 100, coolant: 70, cpu: 80 },
  decay:   { battery: 7,   coolant: 6,  cpu: 8 },
  actions: { charge: 20,   cool: 15,    optimize: 10 }
};
```

> На 5 студент может сделать свой конфиг и выбрать его при старте.

Усложнение B. Лог событий (10 последних)

Хранить лог действий в замыкании и выводить его в DOM (например, в `<ul id="log">`):

- “+10 воды”
- “тик: -8 воды”
- “умер: water=0”

Усложнение C. Пауза/Продолжить

Кнопка “Pause/Play”:

- останавливает тик
- запускает тик снова

Усложнение D. Условия-эффекты

Пример:

- если water < 20 → energy убывает быстрее
- если food < 20 → water убывает быстрее

Усложнение E. Защита от действий после смерти

После смерти методы `drink/eat/rest` не должны менять состояние.

Формат сдачи

1. Загрузить Вашу работу в Ваш GitHub репозиторий с папкой проекта (HTML/CSS/JS).
2. Добавить README.md к проекту со следующими данными:

- персонаж (кто он)
- какие правила убывания/действий
- какие усложнения сделаны (для “5”)

3. Отправить мне ссылку на репозиторий